Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Ростовский государственный экономический университет (РИНХ)»

ОТЧЁТ

по лабораторной работе № 1.2

«Лямбда-выражения»

по дисциплине «Разработка и сопровождение программных систем»

Вариант 13

Выполнил: студент группы ИСТ-331
Хасаев Умахан

Ростов-на-Дону
2026

Репозиторий с исходным кодом: https://github.com/warhogz/software-development-and-maintenance

Цель работы

Создать Java-проект с использованием лямбда-выражений.

Задание

1. Изучить теоретический материал.

2. Создать приложение, реализующее лямбда-выражения в соответствии с описанием, приведённым в лабораторной работе.

3. Разработать приложение с использованием лямбда-выражений по индивидуальному заданию (вариант 13: предметная область «Области – Районы»).

Ход работы

Лямбда-выражение – это краткая запись реализации единственного метода функционального интерфейса. Оно состоит из списка параметров, оператора -> и тела выражения. Лямбды можно хранить в переменных, передавать в методы и возвращать из методов, что обеспечивает отложенное выполнение кода.

1. Примеры из методических указаний. В классе LambdaApp (листинг 1) реализованы: простые и блочные лямбды, несколько лямбд для одного интерфейса, лямбды без возвращаемого значения, использование переменных уровня класса и локальных переменных, обобщённый функциональный интерфейс, передача лямбды в метод, ссылки на статический метод, метод объекта и конструктор, возврат лямбды из метода. Результат работы приведён в листинге 2.

Листинг 1 – Класс LambdaApp

```
package demo;

/** Примеры из методических указаний к лабораторной работе 1.2 */
public class LambdaApp {
```

```
interface Operationable {
    int calculate(int x, int y);
}

interface Operation {
    int calculate();
}

interface Printable {
    void print(String s);
}

interface GenericOperationable<T> {
    T calculate(T x, T y);
}

interface Expression {
    boolean isEqual(int n);
}

interface UserBuilder {
    User create(String name);
}

interface BinaryOperation {
    int execute(int x, int y);
}

static class ExpressionHelper {
    static boolean isEven(int n) {
        return n % 2 == 0;
    }

    static boolean isPositive(int n) {
        return n > 0;
    }

    boolean isOdd(int n) {
        return n % 2 != 0;
    }
}

static class User {
    private final String name;

    User(String n) {
        this.name = n;
    }

    String getName() {
        return name;
    }
}

static int x = 10;
static int y = 20;

public static void main(String[] args) {
    System.out.println("--- Лямбда-выражение ---");
    Operationable operation = (a, b) -> a + b;
    System.out.println(operation.calculate(10, 20)); // 30
```

```
System.out.println("--- То же через анонимный класс ---");
Operationable op = new Operationable() {
    public int calculate(int a, int b) {
        return a + b;
    }
};
System.out.println(op.calculate(20, 10)); // 30


System.out.println("--- Несколько лямбд для одного интерфейса ---");
Operationable operation1 = (int a, int b) -> a + b;
Operationable operation2 = (int a, int b) -> a - b;
Operationable operation3 = (int a, int b) -> a * b;
System.out.println(operation1.calculate(20, 10)); // 30
System.out.println(operation2.calculate(20, 10)); // 10
System.out.println(operation3.calculate(20, 10)); // 200


System.out.println("--- Терминальная лямбда ---");
Printable printer = s -> System.out.println(s);
printer.print("Hello Java!");


System.out.println("--- Переменные уровня класса ---");
Operation op1 = () -> {
    x = 30;
    return x + y;
};
System.out.println(op1.calculate()); // 50
System.out.println(x);               // 30 - значение x изменилось


System.out.println("--- Локальные переменные ---");
int n = 70;
int m = 30;
Operation op2 = () -> {
    // n = 100; - так нельзя
    return m + n;
};
System.out.println(op2.calculate()); // 100


System.out.println("--- Блочная лямбда ---");
Operationable division = (int a, int b) -> {
    if (b == 0)
        return 0;
    else
        return a / b;
};
System.out.println(division.calculate(20, 10)); // 2
System.out.println(division.calculate(20, 0));  // 0


System.out.println("--- Обобщённый функциональный интерфейс ---");
GenericOperationable<Integer> g1 = (a, b) -> a + b;
GenericOperationable<String> g2 = (a, b) -> a + b;
System.out.println(g1.calculate(20, 10));     // 30
System.out.println(g2.calculate("20", "10")); // 2010


System.out.println("--- Лямбда как параметр метода ---");
int[] nums = {1, 2, 3, 4, 5, 6, 7, 8, 9};
Expression func = k -> k % 2 == 0;
System.out.println(sum(nums, func));       // 20
System.out.println(sum(nums, k -> k > 5)); // 30


System.out.println("--- Ссылки на методы ---");
System.out.println(sum(nums, ExpressionHelper::isEven)); // 20
Expression expr = ExpressionHelper::isPositive;
System.out.println(sum(nums, expr));                     // 45
ExpressionHelper helper = new ExpressionHelper();
System.out.println(sum(nums, helper::isOdd));            // 25
```

```
        System.out.println("--- Ссылка на конструктор ---");
        UserBuilder userBuilder = User::new;
        User user = userBuilder.create("Tom");
        System.out.println(user.getName());


        System.out.println("--- Лямбда как результат метода ---");
        BinaryOperation f = action(1);
        System.out.println(f.execute(6, 5));         // 11
        System.out.println(action(2).execute(8, 2)); // 6
    }


    private static int sum(int[] numbers, Expression func) {
        int result = 0;
        for (int i : numbers) {
            if (func.isEqual(i))
                result += i;
        }
        return result;
    }


    private static BinaryOperation action(int number) {
        switch (number) {
            case 1: return (a, b) -> a + b;
            case 2: return (a, b) -> a - b;
            case 3: return (a, b) -> a * b;
            default: return (a, b) -> 0;
        }
    }
}
```

Листинг 2 – Результат работы программы LambdaApp

```
--- Лямбда-выражение ---
30
--- То же через анонимный класс ---
30
--- Несколько лямбд для одного интерфейса ---
30
10
200
--- Терминальная лямбда ---
Hello Java!
--- Переменные уровня класса ---
50
30
--- Локальные переменные ---
100
--- Блочная лямбда ---
2
0
--- Обобщённый функциональный интерфейс ---
30
2010
--- Лямбда как параметр метода ---
20
30
--- Ссылки на методы ---
20
45
25
--- Ссылка на конструктор ---
Tom
--- Лямбда как результат метода ---
11
6
```

2. Индивидуальное задание. Созданы функциональные интерфейсы: DistrictCondition (условие для района), DistrictPrinter (вывод района, ничего не возвращает), DistrictFactory (создание района – ссылка на конструктор), AreaShare (доля площади района), Converter<T, R> (обобщённый) и AreaOperation (операция над площадями) (листинги 3–8).

Листинг 3 – Интерфейс DistrictCondition

```
package variant13;

/** Функциональный интерфейс: условие для района */
public interface DistrictCondition {
    boolean check(District district);
}
```

Листинг 4 – Интерфейс DistrictPrinter

```
package variant13;

/** Функциональный интерфейс для терминальных лямбд (ничего не возвращает) */
public interface DistrictPrinter {
    void print(District district);
}
```

Листинг 5 – Интерфейс DistrictFactory

```
package variant13;

/** Функциональный интерфейс для ссылки на конструктор District::new */
public interface DistrictFactory {
    District create(String name, double area, String adminCenter, String head);
}
```

Листинг 6 – Интерфейс AreaShare

```
package variant13;

/** Доля площади района в площади области, % */
public interface AreaShare {
    double calculate(District district, Region region);
}
```

Листинг 7 – Интерфейс Converter

```
package variant13;

/** Обобщённый функциональный интерфейс */
public interface Converter<T, R> {
    R convert(T value);
}
```

Листинг 8 – Интерфейс AreaOperation

```
package variant13;

/** Операция над двумя площадями */
public interface AreaOperation {
    double execute(double x, double y);
}
```

Классы District и Region (листинги 9–10) используют лямбды как параметры методов: метод forEachDistrict() выполняет действие для каждого района, printIf() выводит районы по условию, totalArea() считает суммарную площадь районов по условию. Класс DistrictChecks (листинг 11) содержит методы для ссылок на статический метод и метод объекта.

Листинг 9 – Класс District

```
package variant13;

/** Район */
public class District {
    private final String name;        // Название
    private final double area;        // Площадь, км²
    private final String adminCenter; // Административный центр
    private final String head;        // Глава

    public District(String name, double area, String adminCenter, String head) {
        this.name = name;
        this.area = area;
        this.adminCenter = adminCenter;
        this.head = head;
    }

    public String getName() { return name; }
    public double getArea() { return area; }
    public String getAdminCenter() { return adminCenter; }
    public String getHead() { return head; }

    @Override
    public String toString() {
        return String.format("%s (адм. центр: %s, площадь: %.0f км², глава: %s)",
                name, adminCenter, area, head);
    }
}
```

Листинг 10 – Класс Region

```
package variant13;

import java.util.ArrayList;
import java.util.List;

/** Область */
public class Region {
    private final String name;        // Название
    private final double area;        // Площадь, км²
```

```
    private final String adminCenter; // Административный центр
    private final String head;        // Глава
    private final List<District> districts = new ArrayList<>();

    public Region(String name, double area, String adminCenter, String head) {
        this.name = name;
        this.area = area;
        this.adminCenter = adminCenter;
        this.head = head;
    }

    public String getName() { return name; }
    public double getArea() { return area; }
    public String getAdminCenter() { return adminCenter; }
    public String getHead() { return head; }
    public List<District> getDistricts() { return districts; }

    public void addDistrict(District district) {
        districts.add(district);
    }

    /** Выполнить действие (лямбду) для каждого района */
    public void forEachDistrict(DistrictPrinter printer) {
        for (District d : districts) {
            printer.print(d);
        }
    }

    /** Вывести районы, удовлетворяющие условию (условие передаётся лямбдой) */
    public void printIf(DistrictCondition condition, DistrictPrinter printer) {
        for (District d : districts) {
            if (condition.check(d)) {
                printer.print(d);
            }
        }
    }

    /** Суммарная площадь районов, удовлетворяющих условию */
    public double totalArea(DistrictCondition condition) {
        double sum = 0;
        for (District d : districts) {
            if (condition.check(d)) {
                sum += d.getArea();
            }
        }
        return sum;
    }

    @Override
    public String toString() {
        return String.format("%s (адм. центр: %s, площадь: %.0f км², глава: %s)",
                name, adminCenter, area, head);
    }
}
```

Листинг 11 – Класс DistrictChecks

```
package variant13;

/** Методы-проверки для ссылок на методы */
public class DistrictChecks {
    private final double limit;

    public DistrictChecks(double limit) {
```

```
        this.limit = limit;
    }

    /** Статический метод: крупный район (от 1500 км²) */
    public static boolean isLarge(District d) {
        return d.getArea() >= 1500;
    }

    /** Нестатический метод: район меньше заданного лимита */
    public boolean isSmaller(District d) {
        return d.getArea() < limit;
    }
}
```

В классе Main (листинг 12) показаны: терминальные лямбды для вывода районов; лямбды как параметры методов; ссылки DistrictChecks::isLarge и smallCheck::isSmaller; ссылка на конструктор District::new; блочная лямбда для расчёта доли площади района; обобщённый интерфейс Converter с разными типами; метод action(), возвращающий лямбду; отложенное выполнение лямбды Runnable, изменяющей поле уровня класса; сортировка районов лямбдой и ссылка на метод System.out::println. Результат работы программы приведён в листинге 13.

Листинг 12 – Класс Main

```
package variant13;

import java.util.ArrayList;
import java.util.List;

/**
 * Лабораторная работа 1.2. Лямбда-выражения.
 * Вариант 13: Области – Районы.
 */
public class Main {

    static int reportCounter = 0; // переменная уровня класса — её можно менять внутри
лямбды

    public static void main(String[] args) {
        // ---------- Ссылка на конструктор ----------
        DistrictFactory factory = District::new;

        Region nn = new Region("Нижегородская область", 76624, "Нижний Новгород",
"Петров П.П.");
        nn.addDistrict(factory.create("Арзамасский район", 1923, "Арзамас", "Иванов
И.И."));
        nn.addDistrict(factory.create("Богородский район", 1300, "Богородск", "Смирнов
А.В."));
        nn.addDistrict(factory.create("Кстовский район", 1203, "Кстово", "Кузнецов
Д.С."));
        nn.addDistrict(factory.create("Павловский район", 1077, "Павлово", "Орлова
Е.Н."));
```

```
        Region vl = new Region("Владимирская область", 29084, "Владимир", "Соколов
В.М.");
        vl.addDistrict(factory.create("Ковровский район", 1999, "Ковров", "Морозов
К.А."));
        vl.addDistrict(factory.create("Суздальский район", 1263, "Суздаль", "Волкова
И.Г."));
        vl.addDistrict(factory.create("Муромский район", 1450, "Муром", "Лебедев
Р.О."));

        // ---------- 1. Терминальные лямбды ----------
        DistrictPrinter printer = d -> System.out.println("   " + d);
        DistrictPrinter shortPrinter = d -> System.out.println("   • " + d.getName() +
" — " + d.getAdminCenter());

        System.out.println("===== 1. Районы областей (терминальные лямбды) =====");
        System.out.println(nn);
        nn.forEachDistrict(printer);
        System.out.println(vl);
        vl.forEachDistrict(shortPrinter);

        // ---------- 2. Лямбды как параметры методов ----------
        System.out.println();
        System.out.println("===== 2. Лямбда как параметр метода =====");
        System.out.printf("   Суммарная площадь всех районов Нижегородской обл.: %.0f
км²%n", nn.totalArea(d -> true));
        System.out.printf("   Площадь районов больше 1250 км²: %.0f км²%n",
nn.totalArea(d -> d.getArea() > 1250));
        String letter = "К"; // локальная переменная (фактически final) используется в
лямбде
        System.out.println("   Районы, адм. центр которых начинается на «" + letter +
"»:");
        nn.printIf(d -> d.getAdminCenter().startsWith(letter), printer);
        vl.printIf(d -> d.getAdminCenter().startsWith(letter), printer);

        // ---------- 3. Ссылки на методы ----------
        System.out.println();
        System.out.println("===== 3. Ссылки на методы =====");
        System.out.println("   Крупные районы (DistrictChecks::isLarge — статический
метод):");
        nn.printIf(DistrictChecks::isLarge, printer);
        vl.printIf(DistrictChecks::isLarge, printer);
        DistrictChecks smallCheck = new DistrictChecks(1300);
        System.out.println("   Районы меньше 1300 км² (smallCheck::isSmaller — метод
объекта):");
        nn.printIf(smallCheck::isSmaller, printer);
        vl.printIf(smallCheck::isSmaller, printer);

        // ---------- 4. Блочная лямбда ----------
        AreaShare share = (district, region) -> {
            if (region.getArea() == 0) {
                return 0;
            }
            return district.getArea() / region.getArea() * 100;
        };
        System.out.println();
        System.out.println("===== 4. Доля площади района в площади области (блочная
лямбда) =====");
        for (District district : nn.getDistricts()) {
            System.out.printf("   %-20s %.2f%%%n", district.getName(),
share.calculate(district, nn));
        }

        // ---------- 5. Обобщённый функциональный интерфейс ----------
        Converter<District, String> toShort = d -> d.getName() + " [глава: " +
d.getHead() + "]";
        Converter<Double, Double> km2ToHa = area -> area * 100;
        Converter<Region, Integer> districtsCount = r -> r.getDistricts().size();
```

```
        System.out.println();
        System.out.println("===== 5. Обобщённый интерфейс Converter<T, R> =====");
        System.out.println("   " + toShort.convert(vl.getDistricts().get(0)));
        System.out.printf("   Площадь «%s»: %.0f га%n", nn.getName(),
km2ToHa.convert(nn.getArea()));
        System.out.println("   Районов во Владимирской обл.: " +
districtsCount.convert(vl));

        // ---------- 6. Лямбда как результат метода ----------
        double nnArea = nn.getArea();
        double vlArea = vl.getArea();
        System.out.println();
        System.out.println("===== 6. Лямбда как результат метода =====");
        System.out.printf("   Сумма площадей областей: %.0f км²%n",
action(1).execute(nnArea, vlArea));
        System.out.printf("   Разница площадей: %.0f км²%n", action(2).execute(nnArea,
vlArea));
        AreaOperation ratio = action(3);
        System.out.printf("   Нижегородская обл. больше Владимирской в %.2f раза%n",
ratio.execute(nnArea, vlArea));

        // ---------- 7. Отложенное выполнение ----------
        Runnable report = () -> {
            reportCounter++;
            System.out.println("   Отчёт №" + reportCounter + ": " + nn.getName()
                    + ", районов: " + nn.getDdistricts().size());
        };
        System.out.println();
        System.out.println("===== 7. Отложенное выполнение =====");
        report.run();
        nn.addDistrict(factory.create("Городецкий район", 2648, "Городец", "Новиков
A.A."));
        report.run(); // та же лямбда, но данные уже изменились

        // ---------- 8. Сортировка лямбдой и ссылка System.out::println ----------
        List<District> all = new ArrayList<>(nn.getDistricts());
        all.addAll(vl.getDistricts());
        all.sort((d1, d2) -> Double.compare(d2.getArea(), d1.getArea()));
        System.out.println();
        System.out.println("===== 8. Все районы по убыванию площади =====");
        all.forEach(System.out::println);
    }

    /** Возвращает лямбда-выражение в зависимости от номера операции */
    private static AreaOperation action(int number) {
        switch (number) {
            case 1: return (x, y) -> x + y;
            case 2: return (x, y) -> Math.abs(x - y);
            case 3: return (x, y) -> y == 0 ? 0 : x / y;
            default: return (x, y) -> 0;
        }
    }
}
```

Листинг 13 – Результат работы программы Main

```
===== 1. Районы областей (терминальные лямбды) =====
Нижегородская область (адм. центр: Нижний Новгород, площадь: 76624 км², глава: Петров
П.П.)
   Арзамасский район (адм. центр: Арзамас, площадь: 1923 км², глава: Иванов И.И.)
   Богородский район (адм. центр: Богородск, площадь: 1300 км², глава: Смирнов А.В.)
   Кстовский район (адм. центр: Кстово, площадь: 1203 км², глава: Кузнецов Д.С.)
   Павловский район (адм. центр: Павлово, площадь: 1077 км², глава: Орлова Е.Н.)
Владимирская область (адм. центр: Владимир, площадь: 29084 км², глава: Соколов В.М.)
   • Ковровский район — Ковров
```

```
   • Суздальский район — Суздаль
   • Муромский район — Муром

===== 2. Лямбда как параметр метода =====
   Суммарная площадь всех районов Нижегородской обл.: 5503 км²
   Площадь районов больше 1250 км²: 3223 км²
   Районы, адм. центр которых начинается на «К»:
   Кстовский район (адм. центр: Кстово, площадь: 1203 км², глава: Кузнецов Д.С.)
   Ковровский район (адм. центр: Ковров, площадь: 1999 км², глава: Морозов К.А.)

===== 3. Ссылки на методы =====
   Крупные районы (DistrictChecks::isLarge — статический метод):
   Арзамасский район (адм. центр: Арзамас, площадь: 1923 км², глава: Иванов И.И.)
   Ковровский район (адм. центр: Ковров, площадь: 1999 км², глава: Морозов К.А.)
   Районы меньше 1300 км² (smallCheck::isSmaller — метод объекта):
   Кстовский район (адм. центр: Кстово, площадь: 1203 км², глава: Кузнецов Д.С.)
   Павловский район (адм. центр: Павлово, площадь: 1077 км², глава: Орлова Е.Н.)
   Суздальский район (адм. центр: Суздаль, площадь: 1263 км², глава: Волкова И.Г.)

===== 4. Доля площади района в площади области (блочная лямбда) =====
   Арзамасский район    2,51%
   Богородский район    1,70%
   Кстовский район      1,57%
   Павловский район     1,41%

===== 5. Обобщённый интерфейс Converter<T, R> =====
   Ковровский район [глава: Морозов К.А.]
   Площадь «Нижегородская область»: 7662400 га
   Районов во Владимирской обл.: 3

===== 6. Лямбда как результат метода =====
   Сумма площадей областей: 105708 км²
   Разница площадей: 47540 км²
   Нижегородская обл. больше Владимирской в 2,63 раза

===== 7. Отложенное выполнение =====
   Отчёт №1: Нижегородская область, районов: 4
   Отчёт №2: Нижегородская область, районов: 5

===== 8. Все районы по убыванию площади =====
Городецкий район (адм. центр: Городец, площадь: 2648 км², глава: Новиков А.А.)
Ковровский район (адм. центр: Ковров, площадь: 1999 км², глава: Морозов К.А.)
Арзамасский район (адм. центр: Арзамас, площадь: 1923 км², глава: Иванов И.И.)
Муромский район (адм. центр: Муром, площадь: 1450 км², глава: Лебедев Р.О.)
Богородский район (адм. центр: Богородск, площадь: 1300 км², глава: Смирнов А.В.)
Суздальский район (адм. центр: Суздаль, площадь: 1263 км², глава: Волкова И.Г.)
Кстовский район (адм. центр: Кстово, площадь: 1203 км², глава: Кузнецов Д.С.)
Павловский район (адм. центр: Павлово, площадь: 1077 км², глава: Орлова Е.Н.)
```

Вывод

В ходе выполнения лабораторной работы изучены лямбда-выражения, функциональные интерфейсы и ссылки на методы. Выполнены примеры из методических указаний и индивидуальное задание для предметной области «Области – Районы». Лямбды позволили передавать условия отбора и действия над районами в методы без создания отдельных классов.